

Lecture 4

Agentic AI as a Process Engineer

Tuning a plasma-etch recipe with an LLM in a closed loop — propose, simulate, measure, revise

Asif Khan Associate Professor · School of ECE and School of MSE
Georgia Institute of Technology · ECE 8803-AI4 · akhan40@gatech.edu

What you'll take away

- 1 **Engineer-in-a-loop:** why an LLM here proposes → measures → revises, and why one-shot answers fail.
- 2 **The etch physics:** SF_6/O_2 chemistry and where anisotropy actually comes from.
- 3 **The digital twin:** what ViennaPS buys you, and emulation vs. full physics.
- 4 **What sets performance:** tool interface, metrology, loss weights, and the baseline — not the prompt.
- 5 **Judging honestly:** beat random search, or admit it's an expensive RNG.

Not a chatbot — an engineer in a loop

Ask once (answer-generator)

“Give me a recipe that etches a 1.5 μm trench.”

The model answers from memory.
It cannot know the numbers — chemistry is nonlinear and geometry-dependent.

Result: confident, wrong, unfalsifiable.



Run the loop (process engineer)

Propose a recipe → simulate on a digital twin →
read the profile → revise the guess.

Every step is checked against ground truth.
The prior over plasma chemistry buys sample efficiency in
the first few runs.

Result: measurable convergence.

Takeaway The value isn't the answer — it's stating a hypothesis and letting measurement falsify it.

A

THE PROBLEM

**Six coupled knobs,
no closed-form inverse**

The task: hit a trench spec you can't solve for

SF₆/O₂ cryogenic-style silicon etch of a masked trench.

One period of a dense array (reflective boundaries), not an isolated feature.

400 nm

trench width

400 nm

inert hard mask

1 μ m

pitch (dense array)

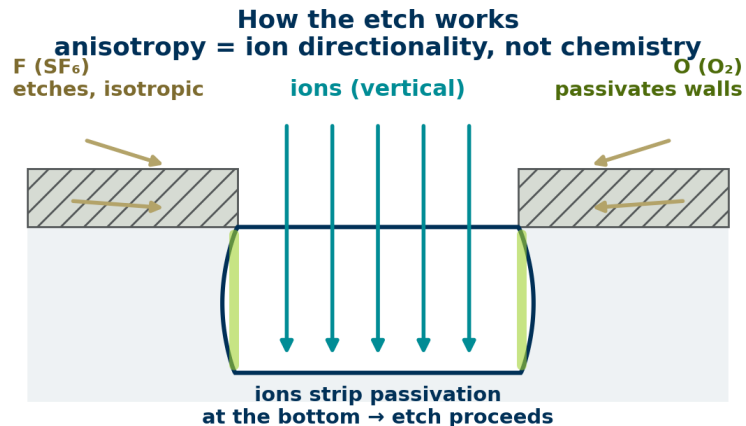
6 knobs

emitted every call

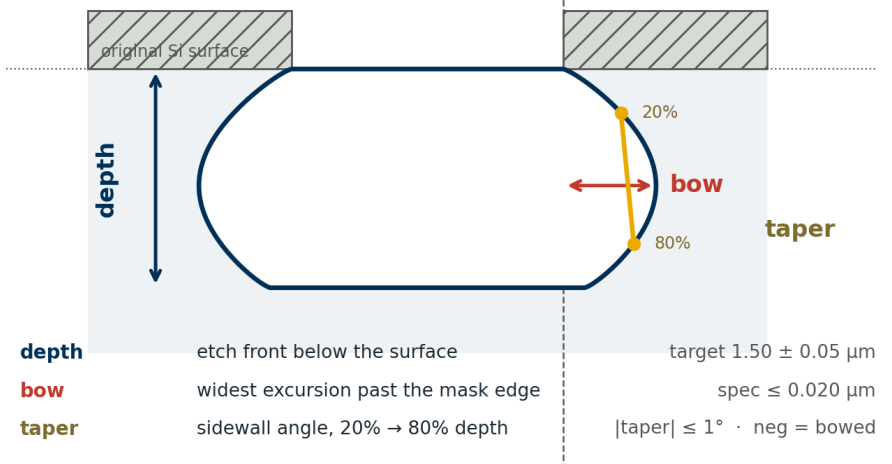
The tension. F etches fast but sideways (bow, mask undercut). O straightens the profile but slows the etch. Every knob moves both metrics, and the coupling is nonlinear — flux at the trench bottom depends on the geometry the etch is creating.

The chemistry — and what we measure

F & O reach every surface — only ions single out the bottom; we grade the trench on depth, bow, taper



What we measure (virtual metrology)



Takeaway Anisotropy is geometric (ions strike the bottom, not the walls). We grade the trench on depth, bow ($\leq 0.020 \mu\text{m}$), and taper.

The action space — six knobs

Knob	Range	What it does physically
etchant_flux	100 – 5000	$\text{SF}_6 \rightarrow$ atomic F. Removes Si, isotropically.
oxygen_flux	1 – 1000	$\text{O}_2 \rightarrow$ atomic O. Sidewall passivation.
ion_flux	1 – 50	How many ions arrive; sets the strip rate.
mean_ion_energy	50 – 300 eV	Sputter yield $\sim \sqrt{E}$ above threshold.
ion_exponent	50 – 1000	Ion angular spread as $\cos^n\theta$. Higher = tighter beam.
duration	0.2 – 1.5 min	Etch time. Nearly a free ride on depth.

Why the search is hard

duration $\approx$ free ride on depth

fix the profile first, then dial time.

O_2 vs ion_exponent

two routes to anisotropy $\rightarrow$ many recipes pass.

The F/O ratio matters

agents that reason in ratios converge far faster.

All six are required every call — “change one thing at a time” is a strategy the agent must choose, not a constraint the interface imposes.

The spec, the loss, and what the agent can see

The spec (loss ≤ 1 per term = in spec)

1.50 ± 0.05

depth (μm)

≤ 0.020

bow (μm)

$\leq 1.0^\circ$

|taper|

Virtual metrology — metrics()

read off the level-set surface like a cross-section SEM:

depth

— etch front below the original surface

taper_deg — wall angle 20→80%; negative = bowed

bow — widest excursion past the mask edge

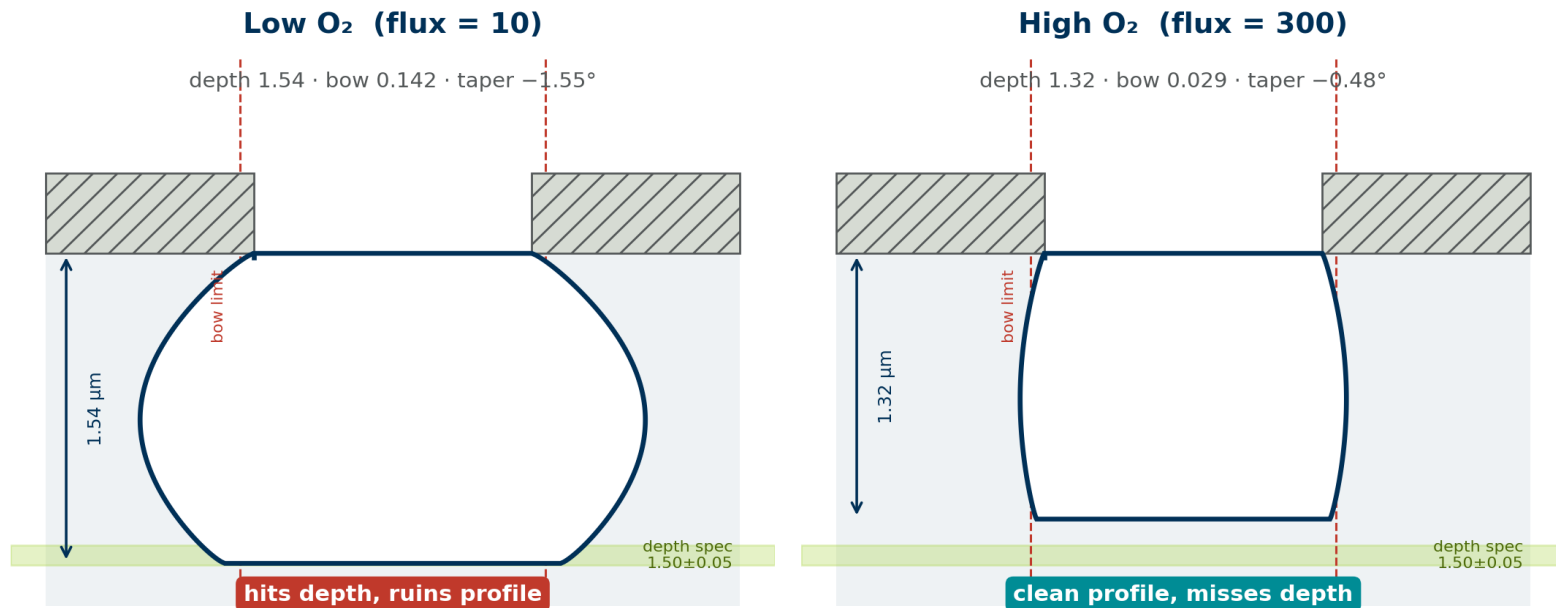
cd_top — CD under the mask; >0.4 = undercut

loss = **depth_err** + **bow_pen** + **taper_pen** — each term normalized; **passed** only when **depth_err** ≤ 1 and **bow_pen** = 0 and **taper_pen** = 0.

Metrology decides what the agent can perceive — bad feature extraction and it optimizes the wrong thing and succeeds at it without complaint. And the loss weights silently decide how much bow you'll trade for depth: an unstated engineering judgment nobody sanity-checks.

The core tension: the passivation trade-off

One knob can't win: O_2 trades depth against profile (both at 0.6 min)

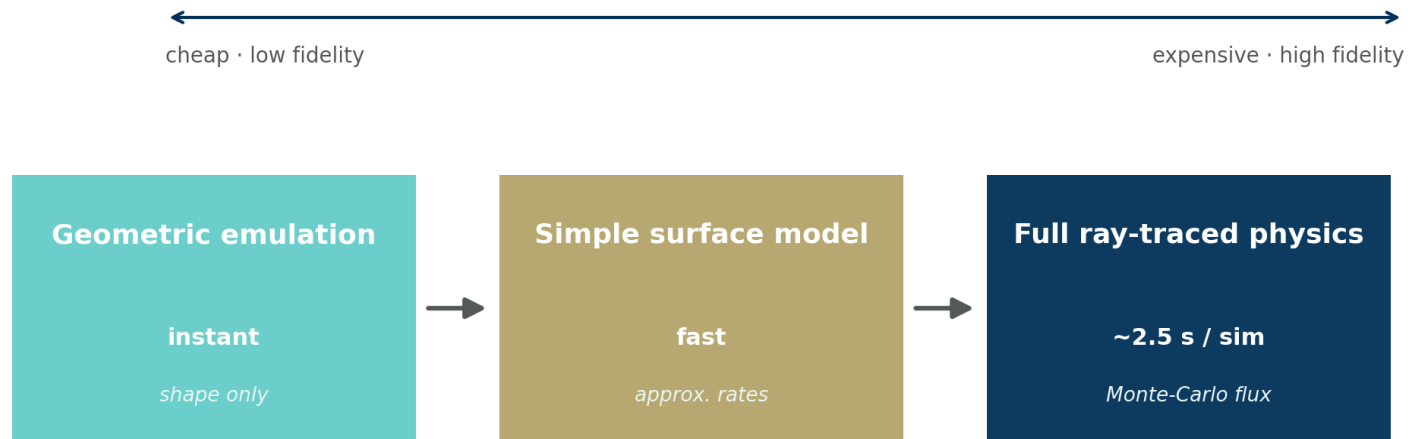


Takeaway Neither profile is acceptable, and neither is fixable with the O_2 knob alone. This is the setup for the whole lab.

The digital twin: ViennaPS

ViennaPS digital twin: one simulator, three fidelities (level-set surface + ray tracing)

~50× cost span



Takeaway Open-source, pip-installable; ~2.5 s per full simulation. The three-fidelity ladder is the setup for the follow-on lab.

ViennaPS — what it is, and who built it

1 A topography simulator. header-only C++ library that models surface evolution during etching, deposition, and oxidation.

2 From TU Wien. built at the Institute for Microelectronics, TU Wien (Vienna University of Technology).

3 Who built it. core contributors Tobias Reiter and Lado Filipovic (with Noah Karnel, Roman Kostal).

4 How to cite it. Reiter & Filipovic, “ViennaPS: A flexible framework for semiconductor process simulation,” SoftwareX (2025).

Lineage & license

Part of the ViennaTools open-source stack — built on:

ViennaLS level sets · **ViennaRay** ray tracing
ViennaHRLE · ViennaCS · ViennaCore

GPU ray tracing since v3.4.0

License: MIT → GPL-3.0 at v4.3.0

Current v4.6.x (2026) — this lab pins 4.6.2

Takeaway An actively developed academic tool — the same class of surface-evolution simulator that sits under TCAD topography.

ViennaPS — where it lives, and how you call it

● GitHub

github.com/ViennaTools/ViennaPS

open source

● PyPI

`pip install ViennaPS`

Python package

● Docs

viennatools.github.io/ViennaPS

API + examples

C++ (header-only) + Python bindings · Linux / macOS / Windows · 2D & 3D · experimental GPU ray tracing

```
import viennaps as vps
vps.setDimension(2) # 2-D cross-section
d = vps.Domain(gridDelta=0.06, xExtent=1.0, boundary=REFLECTIVE)
vps.MakeTrench(d, trenchWidth=0.4, maskHeight=0.4).apply() # geometry
model = vps.SF6O2Etching(ionFlux=12, etchantFlux=1800, oxygenFlux=100, ...)
p = vps.Process(d, model, duration=0.73)
p.setParameters(vps.RayTracingParameters(raysPerPoint=250))
p.apply() # evolve the surface
surface = d.getLevelSets()[-1] # → mesh → depth / bow / taper
```

Takeaway One import, a few builder calls, then `p.apply()` — the whole twin is ~10 lines of Python.

ViennaPS — inputs and outputs

Inputs

Geometry builders: MakeTrench · MakeHole · MakeFin · MakeStack, or import a GDS layout.

Process model

prebuilt (SF₆/O₂ etch, TEOS/plasma deposition, Deal-Grove oxidation) or a custom model.

Parameters

process knobs (fluxes, ion energy & exponent, duration); rays-per-point & seeds; grid resolution; boundary conditions.

Outputs

Evolved surface the level-set representation after the process runs.

Meshes

surface & volume meshes, exportable to VTK for viewing.

Virtual metrology

read off the surface polyline: depth, bow, taper, cd_top — exactly our metrics().

Takeaway Where it stops: topography only — no drift-diffusion, no device I-V. That's where TCAD device simulation begins.

B

THE AGENT

**Propose, simulate,
measure, revise**

Why a single-shot answer is worthless

The task is constructed so the model cannot answer from memory:



The recipe is not knowable a priori

Depth and profile depend on nonlinear chemistry and on the geometry the etch itself creates — there is no lookup answer to recall.



Anything from memory is wrong

A plausible-sounding recipe lands near loss 0.8 at best. It reads confident and is unfalsifiable without a simulator.



The only way through is the loop

Propose → measure against ground truth → revise. Sample efficiency, not first-shot accuracy, is what to optimize.

The agent loop

tool result → revise the hypothesis



Tool `run_etch_process` · Model `claude-sonnet-5` · Budget 12 calls

Takeaway The interface — not the prompt — is doing most of the work.

The tool is the agent's process manual

```
run_etch_process(  
    ion_flux, etchant_flux, oxygen_flux,  
    mean_ion_energy, ion_exponent, duration )  
  
returns →  
    depth, bow, cd_top,  
    taper_deg    # NEGATIVE = bowed  
    ion_exponent # higher = more directional  
    depth_err, bow_pen, taper_pen, passed
```

Two things matter more than the prompt

- 1 **The docstring is the manual.** every knob→effect hint lives in the schema, or must be discovered by simulation.
- 2 **The result is metrology.** we hand back numbers, not advice, and let the model reason.

Exercise 1: strip the hints and re-run. The model didn't get dumber — the interface did.

Takeaway System prompt: “work like an engineer, not a random sampler — baseline → hypothesis → one knob → co-optimize; stop when it passes.”

The instruction, and how it picks the next run

SYSTEM PROMPT

You are a plasma etch process engineer tuning an SF_6/O_2 recipe on a digital twin.

TARGET depth $1.50 \pm 0.05 \mu\text{m}$ · bow $\leq 0.020 \mu\text{m}$ · |taper| $\leq 1.0^\circ$

You have a hard budget of calls.

Work like an engineer, not a random sampler: run a baseline, form a hypothesis about which knob moves which metric, change one thing at a time when the causality is unclear, and only then co-optimize. State your reasoning before each call. Stop the moment a recipe passes.

FIRST USER MESSAGE

“Find a passing recipe. You may call the tool at most 12 times. Start with a baseline you can reason from.”

How it decides the next run

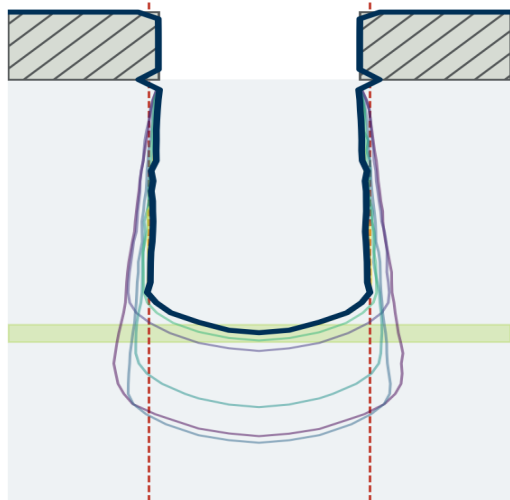
- 1 **Read the metrology.** the tool returns depth · bow · taper · penalties · passed; every prior recipe + result stays in context.
- 2 **State a hypothesis.** in words — “bow too high → raise O_2 or tighten the ion beam.”
- 3 **Emit a full recipe.** all six knobs every call, so “change one thing” is a choice the model makes, not a rule.

The LLM is the search policy. No optimizer sits in the loop — the “algorithm” is the model reasoning over accumulated tool feedback. It stops when passed = true (or the budget runs out).

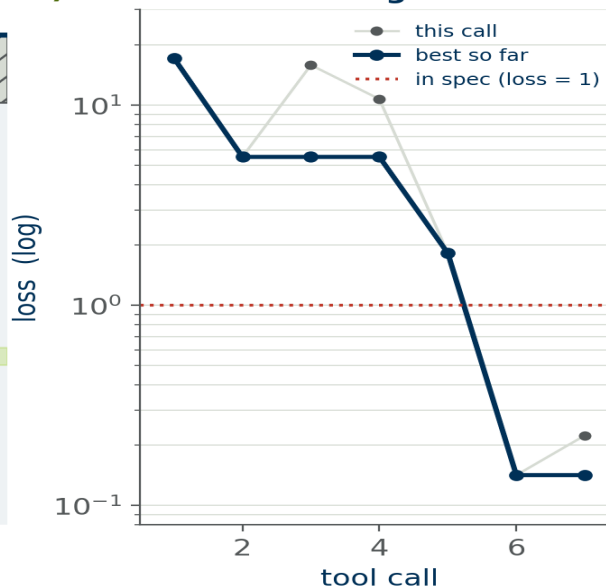
Reading the live view

The live view — the real agent run, redrawn after every tool call

profile trajectory ✓ IN SPEC (call 6)



convergence



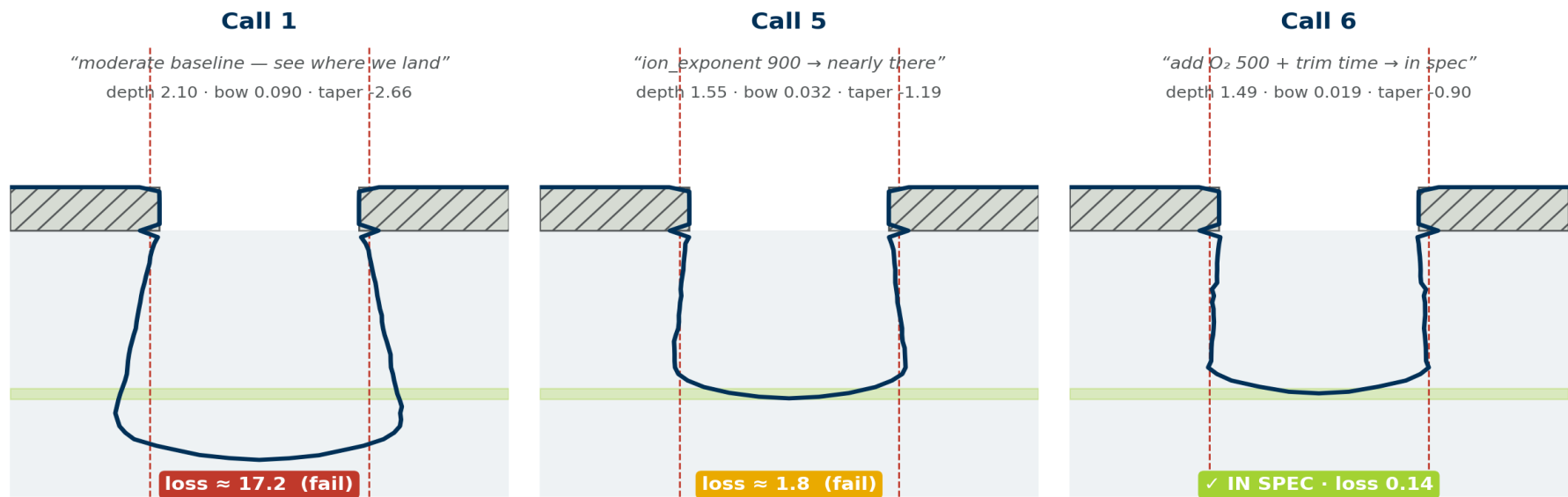
recipe log

call	O ₂	n ⁱ	dur	loss
1	300	300	0.80	17.20
5	300	900	0.53	1.82
6	500	900	0.55	0.14
7	400	1000	0.53	0.22

Takeaway The real agent run from your notebook: 7 tool calls, best loss falls 17.2 → 0.14, crossing into spec at call 6.

A run: hypothesis → consequence

The real agent run — baseline overshoots, then it reasons its way into spec by call 6



Takeaway The agent's real trajectory: baseline overshoots to 2.10 μ m (loss 17.2); ion_exponent 900 nearly nails it by call 5; call 6 adds O₂ and passes (loss 0.14).

The recipe the agent landed

The passing recipe — call 6 of the agent run (all six knobs)

20

ion flux

1500

etchant flux

500

O₂ flux

150 eV

ion energy

900

ion exponent

0.55 min

duration

Measured profile (real run)

1.493 μm

depth ✓

0.0187

bow ✓

-0.90°

taper ✓

0.142

loss — PASS

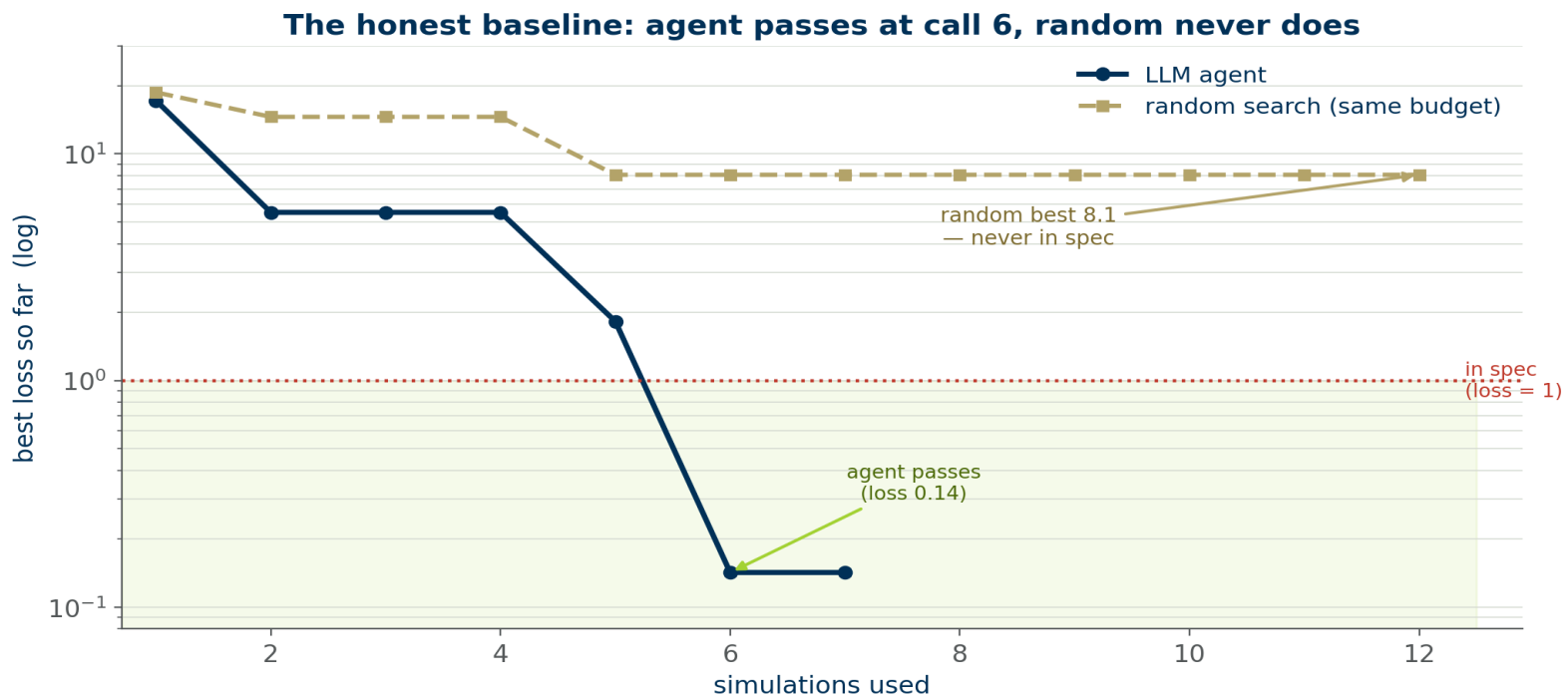
Takeaway Found in 6 of 12 calls — depth, passivation, and ion directionality co-optimized, exactly as the agent reasoned.

C

JUDGING IT HONESTLY

**Beat the baseline,
or admit it's an RNG**

The honest baseline: agent vs. random search



Takeaway Real numbers: the agent reaches spec at call 6 (loss 0.14); random search on the same budget bottoms out at 8.1 and never passes.

What actually determined performance

Agent performance is dominated by things that are not the prompt:

1

Tool interface

What causal information sits in the schema description sets convergence speed.

2

Metrology

metrics() decides what the agent can perceive — bad features, wrong target.

3

Loss function

The weights in score() are an unstated judgment that silently picks the answer.

4

The baseline

Same simulator, same budget, random search — the sanity check on all of it.

What this lab does not claim

This is not evidence that LLMs are good at process optimization. Bayesian optimization will likely beat the agent at $n = 50$.

What the agent actually has: a prior over plasma chemistry and the ability to state a hypothesis in words — which may buy sample efficiency in the first ten runs and nothing after.

That's an empirical question students can settle with the code in front of them — a far better assignment than one where the conclusion is pre-baked.

D

WHERE IT GOES

Exercises, extensions, and the follow-on lab

Exercises

1

Tool design

Strip the causal hints from the schema. How many extra simulations? The model didn't get dumber — the interface did.

2

Stronger baseline

Replace random search with BO. Does the LLM still win at $n=12$? $n=50$? Where's the crossover?

3

Infeasible spec

Demand $cd_top \leq 0.41$ and $depth \geq 2.0 \mu m$. Can the agent say it's unsatisfiable, or hallucinate a fix?

4

Multi-agent

Split proposer / critic (as in AgenticTCAD). Do the extra tokens buy anything?

5

Go 3-D

`setDimension(3)` + `MakeHole`. $\sim 50\times$ per sim. Does sample efficiency justify the wall-clock?

6

Reliability

Five runs at temperature 1. Report spread in iterations-to-spec. Would you put this in a fab?

Where this goes: the follow-on BO lab

Same simulator, a better fit for Bayesian optimization / active learning. Four problem shapes:

- 1 Multi-fidelity** — Three genuine cost levels are already built in ($\sim 50\times$ span). Deciding which fidelity to query is the real fab problem.
- 2 Constrained multi-objective** — Drop the scalar loss: maximize depth, minimize bow, s.t. $|\text{taper}| \leq 1^\circ$. The Pareto front makes the hidden weights explicit.
- 3 Heteroscedastic noise** — Ray tracing gives stochastic observations; noise grows with aspect ratio. Students fit a learned noise model — rarely teachable without faking it.
- 4 Unknown feasibility** — Bad recipes make metrics() return NaN. A GP classifier learns the physically interpretable failure boundary.

Bridge: warm-start BO with the agent's first 5 proposals vs. Sobol init. Does the chemistry prior beat space-filling? (Helps at $n=10$, gone by $n=50$.)

Related work & where to read next

ViennaPS

TU Wien (Reiter & Filipovic). Open-source level-set topography simulator with Python bindings. Ends where TCAD device simulation begins.

AgenticTCAD

Multi-agent framework + fine-tuned TCAD code-gen LLM. Hit IRDS-2024 specs on a 2 nm nanosheet FET in 4.2 h vs. 7.1 days for human experts.

SemiKong

Open-weights semiconductor-tuned LLM (AI Alliance / TEL / FPT), free for academic use. Demoed as an “Etching Advisor” — the key-free path [here](#).

Commercial (mention-only)

Applied Materials Aix · Siemens Fuse EDA agent w/ NVIDIA · Synopsys autonomous CAE. Not student-accessible.

Discussion: *domain-tuned 8B vs. general frontier model on the same tool loop — knowledge in the weights, or reasoning over feedback?*

Takeaways

- 1 **The loop is the method.** An LLM as a process engineer proposes, measures against a digital twin, and revises. One-shot answers are worthless here.
- 2 **Anisotropy is geometric.** F and O reach every surface; only ion directionality singles out the trench bottom. No single knob hits the spec.
- 3 **Interface > prompt.** Tool schema, metrology, and loss weights determine the answer far more than prompt wording.
- 4 **Always keep the baseline.** Beat random search on the same budget, or it's an expensive RNG. What we proved is a question, not a verdict.

Discussion: *domain-tuned 8B vs. general frontier model on the same tool loop — weights or reasoning?*